Practical - 8
Title : Perform data visualization

a. **Perform data visualization using Python on any sales data.**

Theory :

The code provided for the coffee sales visualization follows a structured data science workflow: **Loading**, **Processing**, and **Visualizing**. Below is a detailed breakdown of how each part of the script works and how it transforms the raw data from Coffe_sales.csv into insights.

1. Data Aggregation (The "Grouping" Logic)

The most critical part of the code is the .groupby() function. In sales analysis, raw data contains individual transactions. To see "Total Sales," we must bundle these transactions together based on a specific category.

- **Coffee Type Analysis:** df.groupby('coffee_name')['money'].sum() looks at every row, finds the coffee name (e.g., "Latte"), and adds up all the values in the money column associated with it.
- **Weekday Analysis:** df.groupby(['Weekday', 'Weekdaysort'])['money'].sum() does the same for days of the week. We include Weekdaysort (a numerical value like 1 for Monday, 2 for Tuesday) so that the chart follows a calendar sequence rather than alphabetical order.

2. Visualization Components

The code uses matplotlib, a standard plotting library, to create bar charts. Bar charts are ideal here because they allow for a clear comparison between discrete categories (different drinks or different days).

- **plt.bar()**: This is the core command that draws the bars. The first argument is the labels (X-axis), and the second is the heights (Y-axis).
- **plt.xticks(rotation=45)**: Because coffee names like "Americano with Milk" are long, we rotate the text by 45 degrees so they don't overlap and remain readable.
- **plt.tight_layout()**: This is a helper function that automatically adjusts the padding around the plot so that labels don't get cut off when the image is saved.

3. Understanding the Output

The script generates two specific images:

1. **coffee_revenue.png**: This identifies your **Profit Drivers**. By sorting the values (using .sort_values(ascending=False)), the highest-earning drink appears first. This helps a business decide which products to promote or which ones might need a price adjustment.
2. **weekday_revenue.png**: This identifies **Peak Times**. In retail, knowing that Friday and Saturday have higher revenue than Monday allows for better inventory management (ordering more milk/beans before the weekend) and staffing.

Why use this specific code?

- **Scalability:** This code works whether you have 10 rows of data or 10,000.
- **Automation:** Instead of manually calculating totals in Excel, you can run this script every time you get a new CSV export from a Point of Sale (POS) system to get instant visual updates.
- **Customization:** You can easily change color='skyblue' to any hex code to match a brand's color palette.

Procedure –

by D.C Rupani College Students Only

```
import pandas as pd
import matplotlib.pyplot as plt

# Load the coffee sales data
df = pd.read_csv('Coffe_sales.csv')

# --- 1. Visualization: Total Revenue by Coffee Type ---
# Group by coffee name and sum the money, then sort by revenue (descending)
coffee_revenue = df.groupby('coffee_name')['money'].sum().sort_values(ascending=False).reset_index()

plt.bar(coffee_revenue['coffee_name'], coffee_revenue['money'], color='skyblue')
plt.title('Total Revenue by Coffee Type')
plt.xlabel('Coffee Name')
plt.ylabel('Total Revenue ($)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.savefig('coffee_revenue.png')
plt.close()

# --- 2. Visualization: Total Revenue by Day of Week ---
# Group by Weekday and Weekdaysort to ensure bars are in chronological order
weekday_revenue = df.groupby(['Weekday',
Weekdaysort'])['money'].sum().reset_index().sort_values('Weekdaysort')

plt.bar(weekday_revenue['Weekday'], weekday_revenue['money'], color='orange')
plt.title('Total Revenue by Day of Week')
plt.xlabel('Day of Week')
plt.ylabel('Total Revenue ($)')
plt.tight_layout()
plt.savefig('weekday_revenue.png')
plt.close()

print("Visualizations saved as 'coffee_revenue.png' and 'weekday_revenue.png'.")
```

b. Perform data visualization using PowerBI on any sales data.

Theory :

Step 1 : Data Extraction :

The data extraction is first step of ETL. There are 2 Types of Data Extraction

1. Full Extraction : All the data from source systems or operational systems gets extracted to staging area. (Initial Load)

2. Partial Extraction : Sometimes we get notification from the source system to update specific date. It is called as Delta load.

Source System Performance: The Extraction strategies should not affect source system performance.

Step 2 : Data Transformation :

The data transformation is second step. After extracting the data there is big need to do the transformation as per the target system. Following are some points of Data Transformation.

- Data Extracted from source system is in to Raw format. We need to transform it before loading in to target server.
- Data has to be cleaned, mapped and transformed

For D.C.Ruparel College Students Only

- There are following important steps of Data Transformation :

1. Selection : Select data to load in target
2. Matching : Match the data with target system
3. Data Transforming : We need to change data as per target table structures Real life examples of Data Transformation :
   - Standardizing data : Data is fetched from multiple sources so it needs to be standardized as per the target system.
   - Character set conversion : Need to transform the character sets as per the target systems. (Firstname and last name example)
   - Calculated and derived values: In source system there is first val and second val and in target we need the calculation of first val and second val.
   - Data Conversion in different formats : If in source system date in in DDMMYY format and in target the date is in DDMONYYYY format then this transformation needs to be done at transformation phase.

Step 3 : Data Loading

- Data loading phase loads the prepared data from staging tables to main tables.

Data Visualization from ETL Process

- Power BI Desktop lets you create a variety of visualizations to gain insights from your data. You can build reports with multiple pages and each page can have multiple visuals. You can interact with your visualizations to help analyze and understand your data
- In this task, you create a report based on the data previously loaded. You use the Fields pane to select the columns from which you create the visualizations.

Procedure :

<u>Importing Data from OData Feed</u>

In this task, you'll bring in order data. This step represents connecting to a sales system. You import data into Power BI Desktop from the sample Northwind OData feed at the following URL, which you can copy (and then paste) in the steps below:
http://services.odata.org/V3/Northwind/Northwind.svc/
Connect to an OData feed:
1) From the Home ribbon tab in Query Editor, select Get Data.
2) Browse to the OData Feed data source.
3) In the OData Feed dialog box, paste the URL for the Northwind OData feed.
4) Select OK.
5) In the Navigator pane, select the Orders table, and then select Transform.

<u>ETL Process in Power BI</u>

a) <u>Remove other columns to only display columns of interest</u> - In this step you remove all columns except ProductID, ProductName, UnitsInStock, and QuantityPerUnit. Power BI Desktop includes Query Editor, which is where you shape and transform your data connections. Query Editor opens automatically when you select Edit from Navigator. You can also open the Query Editor by selecting Edit Queries from the Home ribbon in Power BI Desktop. The following steps are performed in Query Editor.
   a) In Query Editor, select the ProductID, ProductName, QuantityPerUnit, and UnitsInStock columns (use Ctrl+Click to select more than one column, or Shift+Click to select columns that are beside each other).
   b) Select Remove Columns > Remove Other Columns from the ribbon, or right-click on a column header and click Remove Other Columns.
   c) Change the data type of the UnitsInStock column - When Query Editor connects to data, it reviews each field and to determine the best data type.
      For the Excel workbook, products in stock will always be a whole number, so in this step you confirm the UnitsInStock column's datatype is Whole Number.
      1. Select the UnitsInStock column.

For D.G.Ruparel College Students Only

2. Select the Data Type drop-down button in the Home ribbon.
3. If not already a Whole Number, select Whole Number for data type from the drop down (the Data Type: button also displays the data type for the current selection).

b) Expand the Order_Details table - The Orders table contains a reference to a Details table, which contains the individual products that were included in each Order. When you connect to data sources with multiples tables (such as a relational database) you can use these references to build up your query. In this step, you expand the Order_Details table that is related to the Orders table, to combine the ProductID, UnitPrice, and Quantity columns from Order_Details into the Orders table.

This is a representation of the data in these tables:

The Expand operation combines columns from a related table into a subject table. When the query runs, rows from the related table (Order_Details) are combined into rows from the subject table (Orders). After you expand the Order_Details table, three new columns and additional rows are added to the Orders table, one for each row in the nested or related table.

1. In the Query View, scroll to the Order_Details column.
2. In the Order_Details column, select the expand icon ( ).
3. In the Expand drop-down:
   a. Select (Select All Columns) to clear all columns.
   b. Select ProductID, UnitPrice, and Quantity.
   c. Click OK.

c) Calculate the line total for each Order_Details row - Power BI Desktop lets you to create calculations based on the columns you are importing, so you can enrich the data that you connect to. In this step, you create a Custom Column to calculate the line total for each Order_Details row. Calculate the line total for each Order_Details row:

1. In the Add Column ribbon tab, click Add Custom Column
2. In the Add Custom Column dialog box, in the Custom Column Formula textbox, enter [Order_Details.UnitPrice] * [Order_Details.Quantity].
3. In the New column name textbox, enter LineTotal.
4. Click OK.

d) Rename and reorder columns in the query -

In this step you finish making the model easy to work with when creating reports, by renaming the final columns and changing their order.

1. In Query Editor, drag the LineTotal column to the left, after ShipCountry.
2. Remove the Order_Details. prefix from the Order_Details.ProductID, Order_Details.UnitPrice and Order_Details.Quantity columns, by double-clicking on each column header, and then deleting that text from the column name.

e) Combine the Products and Total Sales queries

Power BI Desktop does not require to combine queries to report on them. Instead, Relationships between datasets can be created. These relationships can be created on any column that is common to the datasets. We have Orders and Products data that share a common 'ProductID' field, so we need to ensure there's a relationship between them in the model we're using with Power BI Desktop.

Simply specify in Power BI Desktop that the columns from each table are related (i.e. columns that have the same values). Power BI Desktop works out the direction and cardinality of the relationship for you. In some cases, it will even detect the relationships automatically.

In this task, confirm that a relationship is established in Power BI Desktop between the Products and Total Sales queries

Step 1: Confirm the relationship between Products and Total Sales

a) First, we need to load the model that we created in Query Editor into Power BI Desktop. From the Home ribbon of Query Editor, select Close & Load.
b) Power BI Desktop loads the data from the two queries.

For D.P. Pioneer College Students Only

c) Once the data is loaded, select the Manage Relationships button Home ribbon.
d) Select the New… button
e) When we attempt to create the relationship, we see that one already exists!, the ProductsID fields in each query already have an established relationship.
f) Select Cancel, and then select Relationship view in Power BI Desktop.
g) double-click the arrow on the line that connects the to queries, an Edit Relationship dialog appears. No need to make any changes, so select Cancel to close the Edit Relationship dialog.

Step 2 : Create charts showing Units in Stock by Product and Total Sales by Year

1) Drag UnitsInStock from the Field pane (the Fields pane is along the right of the screen) onto a blank space on the canvas. A Table visualization is created. Next, drag ProductName to the Axis box, found in the bottom half of the Visualizations pane. Then we then select Sort By > UnitsInStock using the skittles in the top right corer of the visualization.
2) Drag OrderDate to the canvas beneath the first chart, then drag LineTotal (again, from the Fields pane) onto the visual, then select Line Chart.
3) Next, drag ShipCountry to a space on the canvas in the top right. Because you selected a geographic field, a map was created automatically. Now drag LineTotal to the Values field; the circles on the map for each country are now relative in size to the LineTotal for orders shipped to that country.

For D.G.Ruparel College Students Only